SRM TRP ENGINEERING COLLEGE

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

PROJECT REPORT

COMPARATIVE STUDY OF SYNCHRONOUS AND ASYNCHRONOUS EXECUTION

A Simulation-Based Study of Processes, Communication Delays and Response Mechanisms

SUBMITTED BY	ACADEMIC DETAILS
Yogesh S.	Course: CS3551 – Distributed Computing
Register No.: 814724104177	Project: Synchronous vs Asynchronous Execution

Submitted in partial fulfilment of the academic requirements for the course in Distributed Computing.

2026

ABSTRACT

Distributed applications coordinate multiple processes that may execute independently and communicate through messages. A key design choice is whether processes coordinate their progress synchronously or continue independently using asynchronous communication. This project develops a simulation model to compare both approaches using logical processes, configurable communication delays and response mechanisms. The synchronous model uses coordinated rounds: a process sends or receives messages within a defined phase and advances only after required responses are available. The asynchronous model allows each process to continue independently; messages are delivered after simulated delays and responses are handled when they arrive.

The simulation records message-send time, delivery time, response time, process completion time and message count. It illustrates how coordination can make execution easier to reason about but may introduce waiting, while asynchronous execution can improve overlap but requires careful handling of delayed or missing responses. The report presents objectives, architecture, algorithms, implementation approach, illustrative observations, comparison, limitations and future extensions. The clock and delays are simulation inputs, not measurements from a real network.

KEYWORDS

Synchronous execution, asynchronous execution, distributed processes, message passing, communication delay, response handling and simulation.

REPORT ORGANIZATION

Section	Title
1	Introduction and problem definition
2	Objectives and background concepts
3	System architecture and simulation model
4	Synchronous execution design
5	Asynchronous execution design
6	Implementation approach and algorithms
7	Simulation observations and comparison
8	Conclusion, limitations and future scope
9	References

1. INTRODUCTION AND PROBLEM DEFINITION

A distributed system consists of independent computing entities that communicate to complete a task. Each entity may have its own execution flow, local state and view of events. Since communication takes time and processes may progress at different rates, coordination becomes an important part of system design.

1.1 Synchronous execution

In a synchronous execution model, the system uses defined coordination points or bounded phases. This project represents execution as a sequence of rounds. Each process performs its assigned work, sends messages and waits until the required messages or acknowledgements are received before advancing. The model is straightforward to inspect because progress is grouped into predictable phases.

1.2 Asynchronous execution

In an asynchronous model, there is no fixed upper bound assumed for message delivery or process-step duration. Processes progress independently. A sent message may be delivered later, and the receiver handles it when available. A process does not need to wait for every other process before continuing unrelated work. This provides flexibility but makes response tracking and timeout decisions more complex.

1.3 Problem definition

The project asks: how do synchronous and asynchronous coordination styles behave when multiple simulated processes exchange messages over channels with different communication delays? The same logical workload is executed under both models. Waiting, completion time, message delivery and response handling are compared to understand the effects of coordination.

1.4 Scope

- Model configurable logical processes, each with local tasks and state.
- Represent communication as messages travelling through channels with configurable delays.
- Implement round-based synchronous responses and event-driven asynchronous responses.
- Collect comparable metrics under the same workload and delay assumptions.
- Treat the output as simulation observations, not universal performance guarantees.

2. OBJECTIVES AND BACKGROUND CONCEPTS

2.1 Project objectives

- Create a clear simulation of multiple independent processes.
- Demonstrate how communication delay affects process progress.
- Implement coordinated rounds for synchronous execution.
- Implement independent progress and delayed message handling for asynchronous execution.
- Compare response latency, completion time, waiting and message count.
- Provide a design that can be extended to more processes and delay patterns.

2.2 Core concepts

Process: An independent logical execution unit that maintains local state and performs operations.

Message: A data item sent between processes. It contains a sender, receiver, sequence number, payload and timing information.

Communication delay: Simulated time between sending a message and its delivery. Different messages may use different delays.

Response mechanism: The way a sender learns that a request has been received or processed. This project uses acknowledgements (ACKs) or result messages.

Logical time: A simulation counter used to order events. It is not a physical clock.

2.3 Assumptions

- Processes do not crash during the baseline experiment.
- Messages are eventually delivered in the baseline simulation.
- Each message has a non-negative configured delay.
- Both modes use the same workload and delay configuration.
- The simulator uses deterministic event ordering for reproducible runs.

2.4 Expected learning outcome

Synchronous coordination makes dependencies explicit but can cause a fast process to wait for a slower participant. Asynchronous execution can overlap independent work, but requires request correlation, state tracking and policies for late or absent responses.

3. SYSTEM ARCHITECTURE AND SIMULATION MODEL

3.1 Architecture overview

The simulation contains five components: Process Manager, Process Workers, Message Queue, Delay Scheduler and Metrics Collector. The Process Manager creates processes and selects the execution mode. Each worker executes local tasks and generates messages. The queue stores messages awaiting delivery. The scheduler advances logical time and releases messages at their delivery times. The collector records events and calculates comparison values.

Component	Responsibility
Process Manager	Creates processes, initializes state and starts the selected mode.
Process Worker	Executes local operations and sends or handles messages.
Message Queue	Stores pending messages with sender, receiver and sequence information.
Delay Scheduler	Schedules delivery at send_time + configured_delay.
Metrics Collector	Records send, delivery, ACK and completion timestamps.

3.2 Process and message representation

```
Process = { process_id, local_state, status, pending_responses }
Message = { message_id, sender, receiver, sequence, payload,
            send_time, delivery_time, message_type }
Event = { event_time, event_type, process_id, message_id }
```

3.3 Delay model

For each message m, delivery_time(m) = send_time(m) + delay(m). A fixed-delay experiment uses the same delay for all messages. A variable-delay experiment assigns different values to messages or links. The event scheduler selects the next event with the smallest event time, making experiments repeatable.

3.4 Response flow

A sender transmits a request and records its message identifier. The receiver processes the request when delivered and schedules an ACK or result. The sender matches the response to the original request and updates its pending-response count. In synchronous mode, the round barrier waits for required responses. In asynchronous mode, each response is handled as an event.

4. SYNCHRONOUS EXECUTION DESIGN

4.1 Operating principle

Synchronous execution is implemented as coordinated rounds. At the beginning of a round, each active process performs its local computation and sends required messages. The scheduler delivers messages according to configured delays. Receivers process requests and return acknowledgements. A round completes when all required responses have arrived. Only then does the next round begin.

4.2 Round-based algorithm

```
INITIALIZE all processes and metrics
FOR each round r:
    mark every active process READY
    FOR each active process p:
        execute p's local task for round r
        send required messages and record send time
    WHILE required responses for round r remain:
        advance time to the next delivery event
        deliver due messages
        receiver processes message and schedules ACK/result
        deliver due responses and update pending counts
    mark round r COMPLETE
REPORT metrics
```

4.3 Response mechanism

The synchronous model uses acknowledgements as a coordination signal. A process records which peers must respond during the current round. An ACK is accepted only if it matches an outstanding request. When the outstanding count reaches zero, the process is eligible to finish the round. This makes dependencies explicit.

4.4 Advantages

- Execution phases and dependencies are easier to understand and debug.
- Round boundaries provide a clear point for collecting results.
- Suitable when tasks naturally require group coordination.

4.5 Disadvantages

- A process that finishes early may wait for a slower process.
- A delayed or missing response can hold up the round without a timeout policy.
- Coordination overhead may grow as participating processes increase.

4.6 Example

Assume three processes have response delays of 2, 5 and 3 simulated time units. If the round requires all three acknowledgements, it cannot complete before the slowest required response arrives. Other processes remain at the round barrier even if their work has finished.

5. ASYNCHRONOUS EXECUTION DESIGN

5.1 Operating principle

Asynchronous execution allows each process to proceed independently. A process performs local work, sends a message, records the request as pending and continues with tasks that do not depend on the response. When a message reaches its destination, the receiver processes it and schedules a response. The sender handles that response whenever it is delivered.

5.2 Event-driven algorithm

```
INITIALIZE processes, event queue and metrics
FOR each process p:
    schedule p's initial local task
WHILE events remain:
    select event with earliest event time
    advance logical time to that event
    IF event is a local task:
        execute task; schedule next independent task
        send messages when required
    ELSE IF event is a request delivery:
        receiver processes request; schedule ACK/result
    ELSE IF event is a response delivery:
        match response to request; update process state
REPORT metrics
```

5.3 Response mechanism

Each request receives a unique identifier. The sender stores it in a pending-request map. An ACK or result carries the same identifier, allowing the sender to match it to the correct request. The response handler removes completed requests and updates process state. If an operation depends on a response, only that operation waits; unrelated work can continue.

5.4 Handling late responses

A practical asynchronous system needs a policy for responses that arrive later than expected. The simulator can record a timeout deadline for each request. If the deadline passes first, the request is marked timed out; a later response can be ignored, recorded as late or used to reconcile state depending on the application. A timeout does not prove remote failure; it only means the response was not received within the chosen interval.

5.5 Advantages and challenges

- Independent tasks can overlap with message transmission and remote processing.
- A slow communication path does not automatically stop unrelated processes.
- Requires request identifiers, pending-state tracking and late-response policies.
- Event interleaving can make debugging more difficult.

5.6 Example

With response delays of 2, 5 and 3 time units, each process handles its own response independently. The process with a 2-unit response can continue its next task while the other requests remain pending. The full workload still depends on its actual dependencies, but independent work need not wait at a global barrier.

6. IMPLEMENTATION APPROACH AND ALGORITHMS

6.1 Suggested implementation

The project can be implemented in Python using a priority queue for scheduled events. Each event is stored with its logical time and a stable sequence number. The scheduler repeatedly removes the earliest event, updates the logical clock and invokes the relevant process handler. The same message and delay structures are reused in both modes so the comparison remains fair.

6.2 Core pseudocode

```
def send_message(sender, receiver, payload, now, delay):
    msg_id = next_message_id()
    msg = Message(msg_id, sender, receiver, payload,
                  send_time=now, delivery_time=now + delay)
    event_queue.push(msg.delivery_time, 'REQUEST', msg)
    metrics.record_send(msg)
    return msg_id

def handle_request(message, now):
    result = process_payload(message.payload)
    ack = make_response(message.id, result, now)
    event_queue.push(ack.delivery_time, 'RESPONSE', ack)

def handle_response(response, process):
    if response.request_id in process.pending:
        process.pending.remove(response.request_id)
        metrics.record_response(response)
```

6.3 Metrics

- **Message latency:** delivery_time – send_time.
- **Response time:** response_delivery_time – request_send_time.
- **Total completion time:** time at which the defined workload completes.
- **Waiting time:** time a process is ready but blocked by required coordination.
- **Message count:** total requests and responses exchanged.

6.4 Test scenarios

Scenario A uses equal delays on all paths. Scenario B uses one slow path and two fast paths. Scenario C increases the number of processes. Scenario D introduces a timeout threshold to observe late-response classification. Each scenario should be run in both modes with the same task count, payload sizes and delay configuration.

6.5 Validation checks

- Every response refers to an existing request identifier.
- A synchronous round does not pass its barrier while required responses remain pending.
- Asynchronous processes can execute independent tasks while requests are pending.
- Recorded message latency equals delivery time minus send time.
- Deterministic configurations produce reproducible event traces.

7. SIMULATION OBSERVATIONS AND COMPARISON

7.1 Illustrative experiment

Consider three processes, P1, P2 and P3. Each sends one request to another process. The simulated response delays are 2, 5 and 3 time units. Synchronous mode requires all three responses before the round can finish. Asynchronous mode allows each process to continue independent work as its own response arrives. These values are illustrative inputs, not empirical benchmark measurements.

Measure	Synchronous model	Asynchronous model
Progress rule	Round/barrier based	Event driven; independent progress
Response handling	Required responses gate round completion	Responses handled as they arrive
Effect of slow path	Can delay all participants at barrier	Primarily delays dependent work
Implementation	Simpler phase coordination	Needs correlation and pending-state tracking
Timeout behavior	May block round without policy	Can isolate timeout to affected request
Best fit	Workloads with group-wide dependencies	Workloads with independent or overlapping tasks

7.2 Interpretation

The synchronous model provides a clear completion condition: all required responses for a round must be received. This makes round state easy to inspect, but a slow response can hold back otherwise-ready processes. In the asynchronous model, independent tasks may overlap with communication delays. This can reduce unnecessary waiting when the workload contains independent operations. It does not guarantee that every workload completes faster; dependent tasks must still wait for their required responses.

7.3 Result presentation

A completed implementation should export a trace table containing event time, process ID, event type, message ID and pending-response count. It should report total simulated completion time and accumulated blocked time for each mode. Both modes must use the same workload, delay assignments and completion criteria. Measured values must come from actual simulator output rather than estimates.

7.4 Key finding

The central difference is the coordination policy. Synchronous execution waits at defined coordination points. Asynchronous execution reacts to events and allows independent progress. The more independent work a workload contains, the more opportunity asynchronous execution has to overlap computation with communication.

8. CONCLUSION, LIMITATIONS AND FUTURE SCOPE

8.1 Conclusion

This project presents a simulation-based comparison of synchronous and asynchronous execution using multiple logical processes, communication delays and response mechanisms. The synchronous design uses coordinated rounds and acknowledgements to ensure required responses are received before the next phase begins. The asynchronous design uses an event queue, request identifiers and pending-response tracking so processes can continue independent work while messages are in transit.

The comparison shows the trade-off between predictable coordination and independent progress. Synchronous execution is easier to reason about when tasks require a shared phase boundary, but slow responses can delay the whole round. Asynchronous execution can make better use of independent work, but needs careful state management, response correlation and policies for timeouts and late messages. The suitable model depends on application dependencies and reliability requirements.

8.2 Limitations

- Uses a simplified logical-time simulator rather than a real distributed deployment.
- Baseline experiments assume reliable delivery and no process crashes.
- Does not model operating-system scheduling, CPU contention or real network congestion.
- Illustrative delay values explain behavior and are not real performance measurements.

8.3 Future enhancements

- Add message loss, duplication, retransmission and process-failure scenarios.
- Support configurable timeout, retry and backoff policies.
- Visualize process timelines and message flows using sequence diagrams or a GUI.
- Run repeated experiments with randomized delays and summarize average and percentile latency.
- Extend the model to compare acknowledgement policies and larger process groups.

9. REFERENCES

[1] A. S. Tanenbaum and M. van Steen, *Distributed Systems: Principles and Paradigms*, 2nd ed., Prentice Hall, 2007.

[2] G. Coulouris, J. Dollimore, T. Kindberg and G. Blair, *Distributed Systems: Concepts and Design*, 5th ed., Addison-Wesley, 2011.

[3] M. van Steen and A. S. Tanenbaum, *Distributed Systems*, 3rd ed., 2017.

[4] Python Software Foundation, “heapq — Heap queue algorithm,” Python documentation. https://docs.python.org/3/library/heapq.html

— END OF REPORT —